

IRIS.CREDIT · SECURITY REVIEW

iris.credit

Iris Protocol.

A security review of Iris Protocol.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
iris.credit	Iris Protocol	Omar Minato7namikazi, 0xCiphky ., Ferran Celades, Vlad Zotov, Michael Lett	August 27 – September 25, 2026	September 30, 2026

§ 00

Contents.

31 pages: overview, confidence ranking, scope, 11 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 9 findings	P. 08
	Remediation Review · 2 findings	P. 26
§ 05	Disclaimer	P. 30
§ 06	About Guardian	P. 31

§ 01

Overview.

Guardian reviewed Iris Protocol across 2 review rounds. The final report includes 6 Medium, 3 Low, 2 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	https://github.com/iris-credit/iris-core
CLIENT	METHODOLOGY
iris.credit	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Iris Protocol	895a35ae752b
PREPARED BY	REMEDIATION REVIEW COMMIT
Omar Minato7namikazi, 0xCiphky ., Ferran Celades, Vlad Zotov, Michael Lett	11577d1d5b96
AUDIT WINDOW	
August 27 – September 25, 2026	
FINAL REPORT	
September 30, 2026	

1.1 Confidence Ranking

4 / 5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.



1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
■ Critical	0	0	0	0	0
■ High	0	0	0	0	0
■ Medium	6	0	2	0	4
■ Low	3	0	2	0	1
■ Informational	2	0	2	0	0
All severities	11	0	6	0	5

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Iris Protocol in `https://github.com/iris-credit/iris-core`. The scope includes 994 source lines across 22 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/adapters/AaveV3Adapter.sol	70	98
src/adapters/MorphoBlueAdapter.sol	92	138
src/blm/Blm.sol	28	46
src/blm/WhitelistBlm.sol	35	54
src/interfaces/IBlm.sol	9	17
src/interfaces/IIris.sol	86	95
src/interfaces/IPod.sol	5	8
src/interfaces/IVenueAdapter.sol	6	10
src/interfaces/IWhitelistBlm.sol	5	11
src/Iris.sol	592	921
src/Pod.sol	25	35
src/adapters/AaveV3Adapter.sol	0	98
src/adapters/MorphoBlueAdapter.sol	0	138
src/blm/Blm.sol	0	46
src/blm/WhitelistBlm.sol	0	54
src/interfaces/IBlm.sol	0	17
src/interfaces/IIris.sol	2	95
src/interfaces/IPod.sol	0	8
src/interfaces/IVenueAdapter.sol	0	10
src/interfaces/IWhitelistBlm.sol	0	11
src/Iris.sol	39	975
src/Pod.sol	0	35

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in. Status is where each finding ended up; its own page says how.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	Liquidation Charges Fixed and Floating	Business logic errors	MEDIUM	RESOLVED	p. 8
M-02	Delayed Rebase Falsely Slashes Solver Bond	Business logic errors	MEDIUM	ACKNOWLEDGED	p. 10
M-03	_accrueLegsView() over-accrues surplus on pre-liquidation Aave collateral and _rebase() converts the overage into solver owned yield	Business logic errors	MEDIUM	ACKNOWLEDGED	p. 12
M-04	setAuthorizationWithSig() lets stale signed grants restore revoked operators	Signatures	MEDIUM	RESOLVED	p. 14
M-05	Borrowers can flash-wash tracked collateral to steal solver Aave yield	accounting bypass	MEDIUM	RESOLVED	p. 15
L-01	Off-ledger venue repay can stale Iris debt and let the borrower extract the solver bond	Business logic errors	LOW	ACKNOWLEDGED	p. 17
L-02	Liquidations Lack Execution Bounds	liquidation economics / MEV	LOW	RESOLVED	p. 21
L-03	Self-Liquidation Erases Future Fixed Interest	state synchronization	LOW	ACKNOWLEDGED	p. 23
I-01	First Aave Borrow Can Desync Debt Index	Business logic errors	INFORMATIONAL	ACKNOWLEDGED	p. 24
REMEDIATION REVIEW					
M-01	Post-wipe collateral supply makes the solver bond claimable by the borrower	Business logic errors	MEDIUM	RESOLVED	p. 26
I-01	Resolved loans permanently lose partial collateral withdrawals after the original deadline	Business logic errors	INFORMATIONAL	ACKNOWLEDGED	p. 28

MEDIUM **M-01**

RESOLVED

Liquidation Charges Fixed and Floating

CATEGORY Business logic errors
LOCATION `src/Iris.sol:439-484, 813-832, 890-905`
REVIEW Main Review

— DESCRIPTION

Iris promises borrowers a fixed-rate obligation: the borrower should pay principal plus fixed interest, while the solver bond absorbs venue floating interest above that fixed amount.

That accounting breaks after a venue liquidation. When a liquidator repays venue debt using seized borrower collateral, the repayment can include accrued floating interest. `_rebase()` then reads only the remaining venue debt and clamps or removes the floating leg that was already paid through the liquidation. The paid floating cost is treated as if it never existed, so it is no longer charged against the solver bond.

Root Cause

`_rebase()` derives the position's floating exposure from the live venue debt after liquidation. It does not separately track realized venue funding costs that were already paid by borrower collateral.

As a result, when venue debt is reduced or fully repaid by liquidation, `_rebase()` can reduce `floatingLeg` to the remaining debt and erase the portion of floating interest that was economically funded by the borrower through seized collateral.

Example

Consider a position at maturity with:

- `100` principal
- `10` floating interest
- `5` fixed interest
- collateral worth `121` debt tokens
- enough solver bond to cover the `5` negative spread

In an 86% LLTV Morpho market, the position is liquidatable. A liquidator can repay the full `110` venue debt, seize about `114.82` of collateral including the venue liquidation bonus, and leave about `6.18` collateral in the Pod.

M-01 Liquidation Charges Fixed and Floating

CONTINUED

After `_rebase()`, Iris records zero principal and zero floating interest while preserving the `5` fixed-interest claim. The solver bond remains untouched. Because collateral remains, the borrower must still pay `5` through `repay()` before calling `escape()`.

Ignoring the venue's ordinary liquidation bonus, the borrower funded `110` through seized collateral and another `5` through Iris. The borrower pays `115` despite the promised fixed obligation being `105`. The erased `10` floating interest is shifted away from the solver bond.

Impact

The borrower can overpay relative to the fixed-rate obligation, while the solver and fee recipient can still receive the fixed settlement and the solver can recover an unslashed bond. This contradicts the fixed-rate design, where solver backing is supposed to absorb venue floating costs above the borrower's fixed obligation.

— **RECOMMENDATION**

Track realized venue funding costs separately from outstanding venue debt. When liquidation uses borrower collateral to retire floating interest, preserve that realized floating cost for settlement instead of deleting it during `_rebase()`

— **RESOLUTION**

Status: Resolved.

iris.credit Team: Fixed at <https://github.com/iris-credit/iris-core/pull/27>

<https://github.com/iris-credit/iris-core/pull/27>

MEDIUM **M-02**

ACKNOWLEDGED

Delayed Rebase Falsely Slashes Solver Bond

CATEGORY Business logic errors

LOCATION `src/Iris.sol:653-656, 691-698, 813-830, 849-887`

REVIEW Main Review

DESCRIPTION

Iris calls `_accrueLegs()` before `_rebase()` when updating a position. If Morpho partially liquidates a Pod between Iris calls, Iris calculates interest using the old debt before reading the reduced Morpho debt. Debt already repaid through the Morpho liquidation is therefore treated as outstanding until Iris is updated. When the floating rate exceeds the fixed rate, this creates excess negative spread and can make a healthy solver bond liquidatable.

The Iris borrower can profit because Morpho recognizes the Pod, rather than the Iris borrower, as its borrower and allows arbitrary callers to perform partial liquidations. The borrower can liquidate the Pod, wait without another Iris update, call `liquidateBond()` with a controlled receiver, and then call `escape()`. Calling `rebase()` promptly prevents this, but no onchain deadline or protocol-enforced keeper guarantees that it will be called when the Morpho liquidation happens.

Consider 10,000 USDC principal, a solver-selected 305.67 USDC bond, 90% bond LLTV, a 5% annual fixed rate, and a high-utilization 86% LLTV Morpho market. The bond is ten times the current public BLM minimum of 30.567 USDC: $10,000 \times (30 \times 0.00010184 + 0.0000015) = 30.567$. Iris allows the solver to post any bond at or above this minimum.

At 26 days, 8 hours, and 4 minutes into the 30-day loan, Iris is updated immediately before the Morpho liquidation. Negative net is 273.224847 USDC, below the 275.103000 USDC bond boundary. A price move makes the Pod liquidatable, and the borrower liquidates 90% of its borrow shares. The remaining Morpho position is healthy at approximately 53.84% LTV.

If `rebase()` is called immediately after the Morpho liquidation, negative net two hours later is 273.417762 USDC and the bond remains healthy. If `rebase()` is not called during those two hours, Iris instead records 275.122214 USDC when `liquidateBond()` is called and allows the bond liquidation.

The correct path leaves 32.606187 USDC of solver bond. The delayed path leaves only 15.264286 USDC, pays the borrower a 15.283500 USDC bounty, and reduces the borrower's closing cost by 17.341901 USDC. This equals the additional solver bond consumed. If Iris had been

M-02 Delayed Rebase Falsely Slashes Solver Bond

CONTINUED

updated immediately, the bond would not become liquidatable until 18 hours, 57 minutes, and 18 seconds after the Morpho liquidation. The bug therefore makes it liquidatable 16 hours, 57 minutes, and 18 seconds early.

— RECOMMENDATION

Prevent a delayed rebase from increasing the amount taken from the solver's bond. When `_rebase()` detects that Morpho liquidated part of the position, Iris should not charge the solver as though the repaid debt remained outstanding until the rebase call. Since Morpho's current balance does not show when the liquidation happened, affected positions should be handled separately instead of estimating the missing interest.

— RESOLUTION

Status: Acknowledged.

iris.credit Team: Acknowledged with an additional comment followed by the remediation (PR) link

MEDIUM **M-03**

ACKNOWLEDGED

`_accrueLegsView()` over-accrues surplus on pre-liquidation Aave collateral and `_rebase()` converts the overage into solver owned yield

CATEGORY Business logic errors

LOCATION —

REVIEW Main Review

— DESCRIPTION

`_accrueLegsView()` computes `surplus` from stale stored `(pos.collateral + pos.surplus)` before live venue collateral is synced. `_accrueLegs()` persists that inflated amount.

Then `_rebase()` uses the inflated total in `liquidated = (pos.collateral + pos.surplus) - venueCollateral`, while only clamping `surplus` if `venueCollateral <= pos.surplus`.

That keeps `pos.collateral + pos.surplus` equal to the live venue balance, but the split is wrong after a partial Aave liquidation: yield is accrued on collateral that was already seized. The excess is paid out later as solver `surplus`, and the borrower's remaining principal collateral is reduced instead.

Example:

1. Borrower opens an Aave-backed loan with `collateral = 100`, `surplus = 0`.
2. Aave partially liquidates `40` collateral / `30` debt, leaving `60` live collateral.
3. Before the first Iris sync, Aave income grows `20%`, so live collateral becomes `72`.
4. Any later path starting with `_accrueLegs()` books `20` surplus from the stale `100` base; `_rebase()` then reduces principal to `52`.
5. On close, the solver receives `20` collateral as surplus and the borrower can only escape `52`.
6. Correct accounting was solver `12`, borrower `60`; `8` borrower collateral is silently reassigned.

This issue is Aave specific; Morpho collateral does not accrue via a collateral index

— RECOMMENDATION

When a liquidation is being rebased, recompute `surplus` from live `venueCollateral` instead of preserving the pre-rebase accrual, or subtract the accrued yield attributable to the liquidated collateral slice before storing `pos.surplus`

M- 03 `_accrueLegsView()` over-accrues surplus on pre-liquidation Aave collateral **CONTINUED**
and `_rebase()` converts the overage into solver owned yield

— **RESOLUTION**

Status: Acknowledged.

iris.credit Team: Acknowledged with an additional comment followed by the remediation (PR) link

<https://github.com/iris-credit/iris-core/pull/21>

MEDIUM **M-04**

RESOLVED

setAuthorizationWithSig() lets stale signed grants restore revoked operators

CATEGORY Signatures
LOCATION —
REVIEW Main Review

DESCRIPTION

`setAuthorizationWithSig()` only checks whether a nonce has never been used, then writes the signed boolean directly.

A signer who later revokes an operator with `setAuthorization()` or a newer signed revoke does not invalidate older unused `isAuthorized=true` signatures, so the former operator can replay the stale grant and regain access.

From there it can use the normal auth-gated sinks in `withdrawBond()`, `claim()`, `withdrawCollateral()`, or `escape()` to redirect assets to itself

RECOMMENDATION

Use a monotonic per-authorizer authorization version and advance it on every direct authorization change, so a later revoke invalidates all older signed grants.

RESOLUTION

Status: Resolved.

iris.credit Team: Fixed at <https://github.com/iris-credit/iris-core/pull/22>

<https://github.com/iris-credit/iris-core/pull/22>

MEDIUM **M-05**

RESOLVED

Borrowers can flash-wash tracked collateral to steal solver Aave yield

CATEGORY	accounting bypass
LOCATION	packages/onchain-ethereum-type4/src/__onchain/poolinstance-728a138a4823392c/src/aave-v3-origin-private/src/contracts/protocol/pool/Pool.sol:supply():L118-L138
REVIEW	Main Review

DESCRIPTION

Aave permits anyone to supply collateral on behalf of a Pod. Iris ignores excess live Aave collateral above its tracked `Position.collateral`.

An overcollateralized borrower can supply collateral directly to Aave for the Pod, then withdraw the same amount through `Iris.withdrawCollateral()`. The direct supply temporarily raises the live venue balance, allowing the Iris withdrawal to return the supplied tokens. The Pod's actual Aave collateral remains unchanged, but Iris decreases `pos.collateral`.

Future Aave collateral-index growth on the washed amount is no longer booked as `Position.surplus`, so the borrower can later recover that hidden yield through `escape()` instead of paying it to the solver and fee recipient.

Root Cause

`withdrawCollateral()` and `_rebase()` do not reconcile or classify direct Aave collateral donations before reducing tracked collateral.

When live collateral exceeds Iris accounting:

- `_rebase()` ignores the upward discrepancy.
- `withdrawCollateral()` allows the borrower to withdraw tokens.
- Iris decrements `pos.collateral`.
- the raw Aave aToken balance can remain economically unchanged.

This lets the borrower reduce the accounting base used by `_accrueLegsView()` to compute solver-owned surplus, without reducing venue safety or committing lasting capital.

Attack Path

1. The borrower opens an Aave-backed Iris position with excess collateral.
2. The borrower supplies `D` collateral directly to Aave on behalf of the Pod.
3. The Pod's live Aave collateral increases by `D`, but Iris does not attribute that increase to `pos.collateral`.

M-05 Borrowers can flash-wash tracked collateral to steal solver Aave yield **CONTINUED**

4. The borrower calls `withdrawCollateral(D)` through Iris.
5. Aave returns `D`, leaving the Pod's live venue collateral effectively unchanged from the pre-attack state.
6. Iris still reduces `pos.collateral` by `D`.
7. Over time, the full live Aave collateral balance earns yield, but Iris computes surplus only on the reduced tracked base.
8. On repayment and `escape()`, the borrower receives the yield earned by the untracked collateral base.

Impact

The borrower can divert collateral yield that the fixed-rate quote economics allocate to the solver and fee recipient. The attack can be capital-neutral with flash liquidity, does not reduce venue health, and scales with the washed collateral amount, venue APY, loan term, and number of positions

— RECOMMENDATION

Reconcile/classify live collateral donations before tracked withdrawals, e.g. checkpoint scaled aToken shares and separate principal/donation buckets, or cap withdrawal accounting by the actual Pod live-balance decrease. Accrue surplus from economically attributed live collateral rather than nominal `Position.collateral` after donation-assisted withdrawal.

Before permitting a tracked withdrawal, reconcile live Aave collateral above Iris accounting

PROOF OF CONCEPT <https://github.com/GuardianOrg/SentryRepo-Iris-Protocol---Round-1-01a03e86f010778c9001660a87cd1d3b/blob/refinery/poc-01a03e87-0774-7ba8-bf6f-a7c6ae5b4519/test/refinery-poc/borrowers-can-flash-wash-tracked-collateral-to-steal-solver--93d8c5ad.sol>

— RESOLUTION

Status: Resolved.

iris.credit Team: Fixed at <https://github.com/iris-credit/iris-core/pull/25>

<https://github.com/iris-credit/iris-core/pull/25>

LOW **L-01**

ACKNOWLEDGED

Off-ledger venue repay can stale Iris debt and let the borrower extract the solver bond

CATEGORY Business logic errors

LOCATION —

REVIEW Main Review

— DESCRIPTION

Summary

Iris can keep `pos.debt` materially higher than the real debt owed on the underlying venue.

This happens when venue debt is reduced outside of Iris, either through direct repayment or through a venue liquidation whose collateral loss is hidden or understated. In those cases, `_rebase()` does not fully reconcile the lower live venue debt into Iris storage.

The stale `pos.debt` then continues accruing floating interest through `_accrueLegsView()`. Later, `liquidateBond()` can treat the position as unhealthy based on the inflated internal debt, while resolving the venue side against the smaller live debt reported by the adapter. This can let the borrower slash the solver bond and potentially receive an unjustified liquidation incentive.

Afterward, the borrower can call `escape()` and recover the remaining collateral by paying only the smaller live venue debt.

Root Cause

`_rebase()` compares Iris accounting against live venue balances, but it only reconciles debt reductions when they are associated with visible collateral loss.

If live venue debt falls while collateral remains unchanged, `_rebase()` returns early and leaves `pos.debt` stale.

If collateral loss is visible but understated, `_rebase()` only credits debt reduction up to the value of that collateral delta. This can also leave `pos.debt` above the true venue debt.

This affects both Morpho and Aave:

L-01 Off-ledger venue repay can stale Iris debt and let the borrower extract the solver bond CONTINUED

- Morpho allows a borrower to directly call `repay(onBehalf = pod)`, reducing venue debt while leaving venue collateral unchanged.
- Aave also allows direct repayment on behalf of the Pod.
- Aave entry/refinance flows can introduce small collateral rounding mismatches. When this happens, `_rebase()` may interpret the external repay as a tiny liquidation and only credit a tiny debt reduction, leaving most repaid debt stale.

Attack Path

1. The borrower opens an Iris position and receives the borrowed debt tokens.
2. The borrower uses those tokens to repay the underlying venue directly on behalf of the Pod.
3. The venue debt decreases, but Iris does not fully sync the repayment.
4. `pos.debt` remains materially overstated.
5. Floating interest continues accruing on the stale internal debt.
6. The position eventually appears to create a solver bond shortfall.
7. The borrower calls `liquidateBond()`.
8. `liquidateBond()` slashes the solver bond and may pay the borrower a liquidation incentive.
9. The borrower calls `escape()` and recovers the remaining collateral by paying only the residual live venue debt.

Impact

- Solver bond value can be consumed by debt that no longer exists on the venue.
- The borrower can receive an unjustified liquidation reward.
- The borrower can close the position more cheaply than the fixed-rate trade intended.
- Iris accounting can diverge materially from the underlying venue state.

Variants

1. Morpho direct repayment

The borrower directly repays Morpho debt with `repay(onBehalf = pod)`.

Venue debt decreases, collateral remains unchanged, and `_rebase()` returns before reducing `pos.debt`. The stale internal debt then accrues floating interest and can later be used to slash the solver bond.

2. Aave direct repayment with rounding mismatch

The borrower directly repays Aave debt on behalf of the Pod.

If Iris's expected collateral balance differs slightly from the live Aave balance due to entry or refinance rounding, `_rebase()` may treat the mismatch as a small liquidation. It

L - 01 Off-ledger venue repay can stale Iris debt and let the borrower extract the solver bond CONTINUED

then credits debt reduction only up to the value of that tiny collateral delta, leaving most of the externally repaid debt in `pos.debt`.

3. Morpho liquidation with collateral returned to the Pod

The same issue can occur through an actual Morpho liquidation.

A liquidator can reduce the Pod's Morpho debt and return the seized collateral to the Pod during the liquidation callback. From Iris's perspective, collateral appears unchanged while debt is lower. `_rebase()` therefore skips the debt reduction and leaves stale `pos.debt`.

4. Partial venue liquidation followed by price movement

A partial venue liquidation can reduce both collateral and debt. If the oracle price drops before `_rebase()` runs, the computed value of the collateral loss is lower.

As a result, `_rebase()` syncs the collateral loss but only credits debt reduction up to the later, lower value of the collateral delta. This can leave `pos.debt` above the live Morpho debt. The phantom principal then accrues floating interest and can later be used to slash the solver bond.

5. Morpho full-debt liquidation masked by collateral resupply

If a loan is opened at or near LLTV, a small interest tick can make the Pod externally liquidatable even though Iris still records the original principal.

The borrower, or a cooperating liquidator, can liquidate all Morpho borrow shares and then resupply the seized collateral to the Pod. Live venue debt becomes zero, while live collateral again matches Iris's stored collateral expectation.

Because `_rebase()` sees no collateral shortfall, it skips the debt reduction. Iris keeps phantom principal in `pos.debt`, which continues accruing floating interest until `liquidateBond()` can slash the solver bond.

6. Aave full-wipe masking variant

After Aave wipes a Pod to zero collateral and zero debt, the borrower can directly resupply collateral on behalf of the Pod up to Iris's stale expected collateral balance.

Because `AaveV3Adapter` reports the raw aToken balance, `_rebase()` sees no collateral shortfall and returns before reconciling the zero live debt. This preserves stale debt, floating loss, and the inflated bond requirement.

The borrower can then call `liquidateBond()` against the stale negative net position, collect the bond incentive, and later recover the injected collateral through `escape()`

L - Off-ledger venue repay can stale Iris debt and let the borrower extract **CONTINUED**
01 the solver bond

— **RECOMMENDATION**

Make `_rebase()` fully reconcile external venue debt reductions, even when there is no collateral loss

— **RESOLUTION**

Status: Acknowledged.

iris.credit Team: Acknowledged, severity disputed to Low. Direct venue repays are documented as out of scope and left unreconciled (Iris.sol header 100-102, 116-118). The bond liquidation path only opens if the variable rate stays above the fixed rate long enough for the stale debt to eat through the bond, which the borrower cannot control, and the direct repayment is lost if it does not.

LOW **L-02**

RESOLVED

Liquidations Lack Execution Bounds

CATEGORY	liquidation economics / MEV
LOCATION	packages/iris-core/src/Iris.sol:withdrawCollateral():L571-L605
REVIEW	Main Review

— DESCRIPTION

Iris's `liquidate()` function provides no maximum repayment, minimum collateral output, or deadline. An authorized borrower can therefore front run a pending overdue liquidation and materially worsen its execution without making it revert.

After `maturity + overduePeriod`, `withdrawCollateral()` reserves tracked collateral only against principal and accrued fixed obligations. It excludes `badBond` and the liquidation incentive. On Aave, accrued collateral yield is recorded as surplus belonging to the solver. This surplus supports venue health but is unavailable to the liquidator. The borrower can therefore withdraw tracked principal to Iris's LLTV boundary while the venue remains healthy.

The pending liquidation still charges `pos.debt + pos.fixedLeg + badBond`, but collateral output is silently capped at the reduced `pos.collateral`. A transaction that was profitable when submitted can execute at a principal loss. The borrower keeps the withdrawn collateral while the solver retains the surplus.

The loss results from borrower controlled state changes between transaction submission and execution, rather than from a liquidator knowingly accepting an unprofitable position. Because `liquidate()` has no caller supplied execution bounds, the transaction proceeds under materially worse terms instead of reverting.

— RECOMMENDATION

Add `maxRepaid`, `minSeized`, and `deadline` parameters to `liquidate()`. Validate them after accrual and rebase but before pulling funds from the liquidator. Optionally disable collateral withdrawals after liquidation eligibility or reserve collateral against the complete liquidation exposure.

PROOF OF CONCEPT <https://github.com/GuardianOrg/SentryRepo-Iris-Protocol---Round-1-01a03e86f010778c9001660a87cd1d3b/blob/refinery/poc-01a03e87-0774-7ba8-bf6f-a7c6ae5b4519/test/refinery-poc/overdue-withdrawal-sandwiches-make-liquidators-pay-borrowers-8eb8fa84.sol>

L-02 Liquidations Lack Execution Bounds

CONTINUED

— RESOLUTION

Status: Resolved.

iris.credit Team: Fixed at <https://github.com/iris-credit/iris-core/pull/23>

<https://github.com/iris-credit/iris-core/pull/23>

LOW **L-03**

ACKNOWLEDGED

Self-Liquidation Erases Future Fixed Interest

CATEGORY state synchronization
LOCATION packages/iris-core/src/Iris.sol:439-485, 813-830, 890-899
REVIEW Main Review

— DESCRIPTION

Iris.repay accrues, rebases a two-sided venue liquidation, then settles the fixed overlay. `_rebase` subtracts venue-repaid principal from `pos.debt`; `_settleLegs` computes residual fixed interest through maturity only on the reduced debt. A borrower-controlled address can trigger permissionless Morpho liquidation while the overlay remains live, then repay Iris and bypass the fixed obligation on the liquidated principal.

Impact: The borrower group can capture the liquidation proceeds while avoiding the remaining fixed interest on the retired principal. The correct economic comparison uses actual debt-token debits: the normal Iris repayment versus the Morpho liquidation payment plus the attacked Iris repayment, with external fees and gas included. In the prompt early-liquidation witness, accrued floating interest is negligible, so the normal payment exceeds the attacked path by almost the remaining fixed face. The solver loses the corresponding contracted return, and the protocol loses its performance fee on that return.

— RECOMMENDATION

Preserve the fixed-term obligation on principal recognized as repaid by an underlying venue liquidation: track venue-repaid principal separately or add residual fixed interest through maturity before reducing `pos.debt`. Add partial/full Aave and Morpho liquidation-then-repay tests to ensure fixed charges are invariant to venue liquidation ordering.

PROOF OF CONCEPT https://github.com/GuardianOrg/SentryRepo-Iris-Protocol---Round-1-01a03e86f010778c9001660a87cd1d3b/blob/venue_self_liquidation_erases_fixed_interest_before_maturity_poc/test/refinery-poc/venue-self-liquidation-erases-fixed-interest-before-maturity-8f501d8c.sol

— RESOLUTION

Status: Acknowledged.

iris.credit Team: Acknowledged, severity disputed to Low. The fixed residual is reserved in `withdrawCollateral`, so a borrower cannot reach the venue liquidation threshold without an exogenous price move of at least the residual, not atomically exploitable.

INFORMATIONAL I-01

ACKNOWLEDGED

First Aave Borrow Can Desync Debt Index

CATEGORY	Business logic errors
LOCATION	packages/iris-core/src/Iris.sol:400-431; packages/iris-core/src/adapters/AaveV3Adapter.sol:103-109
REVIEW	Main Review

— DESCRIPTION

Iris snapshots Aave's normalized variable-debt index before entering the venue. If a reserve has zero scaled variable debt, a positive stored `currentVariableBorrowRate`, and a meaningful idle interval, Aave's view function projects an index above the stored index. This state can occur when the reserve has a nonzero configured base variable rate.

During the first subsequent borrow, Aave does not persist the projected index because scaled variable debt is still zero when the reserve indexes are updated. The Pod's debt is therefore minted using the lower stored index, while Iris has already recorded the higher projected index.

Iris consequently stores a debt-index baseline above the index used to mint the Pod's debt. Accrue-first actions revert until the live index catches up. After catch-up, Iris omits the intervening venue interest even though `repay()` must still fund the full live Aave debt. In an affected configuration, the difference could be drawn from pooled assets backing unrelated bonds or claims.

This is a latent configuration risk, not a currently executable production loss. The deployed Iris BLMs admit USDC, USDT, and WETH debt. Their Aave reserves have nonzero aggregate debt and reset to a zero base rate if debt reaches zero, preventing the required idle-index projection.

The PoC uses tBTC and a configured dormant-reserve state to demonstrate the underlying index behavior. That configuration cannot currently be originated through deployed Iris. The condition would become relevant only if Iris later admits an Aave reserve with zero scaled variable debt and a positive stored `currentVariableBorrowRate`.

— RECOMMENDATION

No immediate remediation is required for the currently supported markets. Reassess this condition before admitting any additional Aave debt asset, particularly a reserve with zero scaled variable debt and a positive stored `currentVariableBorrowRate`.

I-01 First Aave Borrow Can Desync Debt Index

CONTINUED

PROOF OF CONCEPT <https://github.com/GuardianOrg/SentryRepo-Iris-Protocol---Round-1-01a03e86f010778c9001660a87cd1d3b/blob/refinery/poc-01a03e87-0774-7ba8-bf6f-a7c6ae5b4519/test/refinery-poc/pre-entry-aave-debt-index-lets-the-first-borrower-drain-pool-c424cd6b.sol>

— RESOLUTION

Status: Acknowledged.

iris.credit Team: Acknowledged with an additional comment followed by the remediation (PR) link

<https://github.com/iris-credit/iris-core/pull/24>

MEDIUM **M-01**

RESOLVED

Post-wipe collateral supply makes the solver bond claimable by the borrower

CATEGORY Business logic errors
LOCATION `src/Iris.sol:840-885`
REVIEW Remediation Review

— DESCRIPTION

The M-01 fix added logic to `_rebase()` that refunds the borrower when collateral sold during a venue liquidation repays more than the recorded principal and fixed interest. Fully wiped positions are excluded from this refund, but Iris recognizes a wipe only when the venue's current collateral and debt balances are both exactly zero.

Following a complete Morpho liquidation, any liquidator can resupply one unit of the seized collateral on behalf of the Pod. Iris consequently observes positive collateral and zero debt. This prevents the wipe from being recognized while leaving `bondRequirement` active.

When `rebase` is called, Iris accounts for the liquidated collateral as repayment, settles the fixed leg, and credits any remaining excess from the solver bond to the borrower. Without the additional collateral, the same liquidation resolves the position without creating a borrower refund.

Any liquidator can supply the collateral and call `rebase` during Morpho's liquidation callback, forcing the bond transfer without authorization from the borrower or solver. The resulting claim belongs to the borrower. If the liquidator is controlled or authorized by the borrower, it can also claim the refund during the callback and use it toward the repayment Morpho collects afterward.

Depending on the liquidated collateral value and accumulated floating interest, the borrower can receive up to the position's full remaining bond. The loss is limited to each affected position but can occur independently across multiple loans.

— RECOMMENDATION

Ensure that collateral supplied after a terminal venue liquidation cannot change how Iris resolves the position or make the solver bond refundable.

M-01 Post-wipe collateral supply makes the solver bond claimable by the borrower

CONTINUED

— RESOLUTION

Status: Resolved.

iris.credit Team: <https://github.com/iris-credit/iris-core/commit/0ab545b8657c57864e5d73566e6da416f15e20b7>

INFORMATIONAL I-01

ACKNOWLEDGED

Resolved loans permanently lose partial collateral withdrawals after the original deadline

CATEGORY Business logic errors

LOCATION `src/Iris.sol:613, withdrawCollateral(), liquidateBond(), escape()`

REVIEW Remediation Review

— DESCRIPTION

While the loan is active, the borrower may withdraw part of the collateral.

The code permits partial withdrawals only until the loan's deadline (maturity plus grace). After that deadline, an unresolved loan can be liquidated, so allowing collateral withdrawals could let the borrower strip value before liquidation.

However, a loan can also become resolved. Bond liquidation ends the solver's side, zeroes the borrower's Iris debt and interest, and leaves the borrower with the underlying venue position at a variable rate. In that resolved state, Iris's `liquidate()`, `liquidateBond()`, `repay()`, and `rebase()` revert. The underlying venue debt can still exist and remains subject to the venue's own rules.

The bug is that `withdrawCollateral()` still applies the original fixed-loan deadline check at line 613 without checking whether the loan is resolved.

Once that deadline passes, even a healthy resolved position can never make a partial collateral withdrawal again, although the fixed-rate Iris obligation has ended and Iris liquidation is no longer possible.

The only remaining route is `escape()`, which requires funding the entire live venue debt and closes the whole position. Recovery is therefore all or nothing.

Impact: The borrower loses the ability to manage the continuing venue position through partial withdrawals and must fully repay and exit to recover any collateral. This is a persistent functional restriction caused by applying a guard against Iris liquidation to a state in which that liquidation cannot occur.

Reproduction

1. Open a loan and resolve it through bond liquidation while a healthy collateralized venue position remains

I-01 Resolved loans permanently lose partial collateral withdrawals after the original deadline CONTINUED

2. Advance beyond the original maturity-plus-grace deadline
3. Attempt an otherwise valid partial collateral withdrawal
4. Observe that the deadline check reverts despite resolution; recovering collateral through `escape()` requires fully repaying the live venue debt and exiting

Summary

collateral withdrawals retain the fixed-loan deadline after resolution into a variable-rate position

Healthy borrowers can lose partial-withdrawal access and need full debt repayment to exit

— RECOMMENDATION

Apply the maturity-plus-grace withdrawal deadline only to unresolved loans. Preserve borrower authorization and the existing collateral and venue health constraints for withdrawals from resolved positions.

— RESOLUTION

Status: Acknowledged.

iris.credit Team: <https://github.com/iris-credit/iris-core/commit/b13a527dff9d6c5b0a25357377841e9f9b152c6a>

just add comments.

Deadline represents the period during which Iris manages the loan. Therefore, after the deadline, the borrower is expected to exit through `escape()`.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.